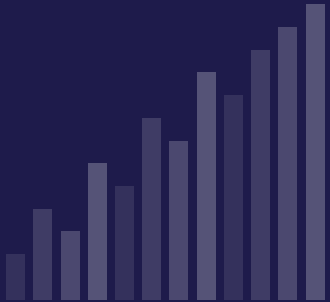
COMPLEMENTO 4 · BLOCO COMPLEMENTAR: ORDENAÇÃO AVANÇADA (OPCIONAL)

Ordenação em tempo linear: contagem e radix

Cormen et al., *Algoritmos*: seções 8.2 e 8.3



TEORIA

Roteiro da aula

Ao final, você será capaz de

- Ordenar inteiros em $0..k$ por contagem em $\Theta(n+k)$
- Explicar por que a estabilidade é essencial para o radix sort
- Ordenar d dígitos em $\Theta(d(n+k))$ e escolher a base
- Medir tempo por elemento: constante no radix, $\lg n$ no `qsort`

Teoria

Counting sort, estabilidade, radix sort LSD e escolha da base

Prática em C

Contagem e radix em C

Experimento

Radix (base 256) contra `qsort` até 4 milhões de elementos

Conexões

GPUs, bancos de dados, bioinformática e processamento de imagens



Teoria

- Cormen et al., *Algoritmos*: seções 8.2 e 8.3
- Ordenação por contagem
- Contagem passo a passo
- Radix sort: dígito por dígito
- Por que a estabilidade importa

TEORIA

Ordenação por contagem

- Nenhuma comparação entre chaves: o limite $\Omega(n \lg n)$ não se aplica.
- Custo $\Theta(n + k)$: linear quando $k = O(n)$.
- O último laço anda **de trás para frente**: iguais saem na ordem original. O algoritmo é **estável**.

COUNTING-SORT(A, B, k)

```
1  crie C[0..k] com zeros
2  para j ← 1 até n faça
3      C[A[j]] ← C[A[j]] + 1
4                          // C[i] = quantos iguais a i
5  para i ← 1 até k faça
6      C[i] ← C[i] + C[i − 1]
7                          // C[i] = quantos ≤ i
8  para j ← n descendo até 1 faça
9      B[C[A[j]]] ← A[j]
10     C[A[j]] ← C[A[j]] − 1
```

TEORIA

Contagem passo a passo

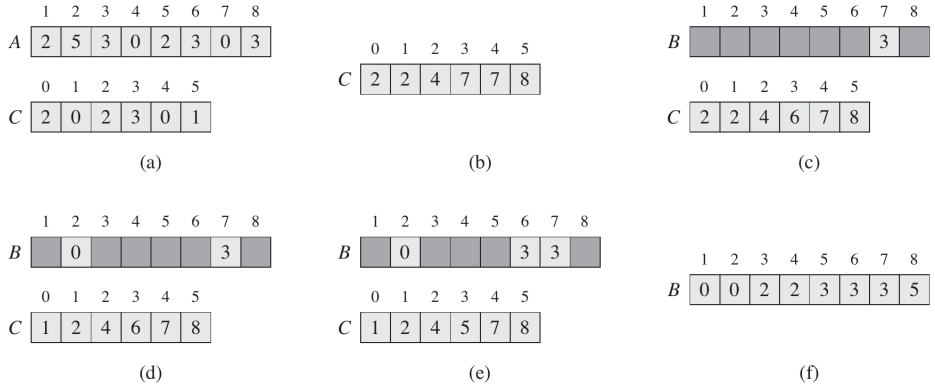


Figura 8.2 de Cormen et al. (2009) — (a) contagens; (b) somas acumuladas; (c)–(e) os três últimos elementos vão para B; (f) resultado.

TEORIA

Radix sort: dígito por dígito

RADIX-SORT(A, d): para $i = 1$ até d, ordene A pelo dígito i com uma ordenação **estável**.

Lema 8.3: d dígitos em base k: $\Theta(d(n + k))$.

Lema 8.4: chaves de b bits em grupos de r bits: $\Theta(\frac{b}{r}(n + 2^r))$. Com $r \approx \lg n$: $\Theta(bn/\lg n)$.

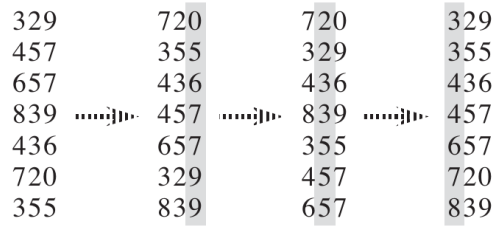


Figura 8.3 de Cormen et al. (2009) — do dígito menos significativo para o mais significativo.

TEORIA

Por que a estabilidade importa

- Depois de ordenar pelo dígito i, os empates nesse dígito devem **manter** a ordem dada pelos dígitos anteriores (menos significativos).
- Invariante: após a passada i, o vetor está ordenado pelos i dígitos menos significativos.
- Uma ordenação instável (quicksort, heapsort) quebraria o invariante.
- **Base na prática:** inteiros de 32 bits em base 256 ($r = 8$): 4 passadas de contagem com vetor C de 256 posições, que cabe no cache.

Radix sort é linear, mas não está sempre à frente: precisa de memória extra $\Theta(n)$, e cada passada percorre o vetor todo.

Prática em C

- Contagem para chaves pequenas; radix em base 10 para as maiores
- Laço final de trás para frente: estabilidade
- Testes com as entradas das Figuras 8.2 e 8.3

PRÁTICA EM C

Contagem e radix sort em C (1/4)

```c
#include <stdio.h>
#include <stdlib.h>

/* COUNTING-SORT: chaves em 0..k. C[v] conta; depois vira "quantos são <= v".
   Percorrer A de trás para frente torna o algoritmo estável. */
void contagem(const int A[], int B[], int n, int k) {
    int *C = calloc(k + 1, sizeof(int));
    for (int j = 0; j < n; j++) C[A[j]]++;
    for (int v = 1; v <= k; v++) C[v] += C[v - 1];
    for (int j = n - 1; j >= 0; j--) B[--C[A[j]]] = A[j];
    free(C);
}
```

PRÁTICA EM C

Contagem e radix sort em C (2/4)

```c
/* RADIX-SORT (LSD) em base 10: contagem estável dígito a dígito, do menos
   significativo para o mais significativo. */
void radix(int A[], int n, int digitos) {
    int *B = malloc(n * sizeof(int));
    for (int d = 0, pot = 1; d < digitos; d++, pot *= 10) {
        int C[10] = {0};
        for (int j = 0; j < n; j++) C[A[j] / pot % 10]++;
        for (int v = 1; v < 10; v++) C[v] += C[v - 1];
        for (int j = n - 1; j >= 0; j--) B[--C[A[j] / pot % 10]] = A[j];
        for (int j = 0; j < n; j++) A[j] = B[j];
    }
    free(B);
}
```

</> PRÁTICA EM C

Contagem e radix sort em C (3/4)

```
27
28 int main(void) {
29     int n;
30     if (scanf("%d", &n) != 1 || n <= 0) return 1;
31     int *A = malloc(n * sizeof(int)), *B = malloc(n * sizeof(int)), max = 0;
32     for (int k = 0; k < n; k++) {
33         if (scanf("%d", &A[k]) != 1 || A[k] < 0) return 1;
34         if (A[k] > max) max = A[k];
35     }
36     if (max < 100) {                          // chaves pequenas: contagem direta
37         contagem(A, B, n, max);
```

PRÁTICA EM C

Contagem e radix sort em C (4/4)

```
        printf("contagem:");
    } else {                                  // chaves maiores: radix nos dígitos
        int digitos = 0;
        for (int m = max; m > 0; m /= 10) digitos++;
        radix(A, n, digitos);
        for (int k = 0; k < n; k++) B[k] = A[k];
        printf("radix:");
    }
    for (int k = 0; k < n; k++) printf(" %d", B[k]);
    printf("\n");
    free(A); free(B);
    return 0;
}
```



Experimento

- Radix LSD em base 256 (4 passadas)
- `qsort` da biblioteca C
- Tempo por elemento, $n = 2^{16}$ a 2^{22}

EXPERIMENTO

Linear contra $n \lg n$, em tempo real

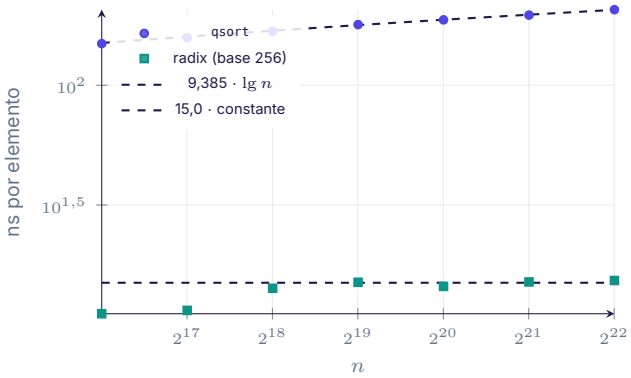
Hipótese. Se $T(n) = cn$, então $T(n)/n = c$: o tempo **por elemento** do radix sort deve ser **constante**. Para o `qsort`, $T(n)/n = c' \lg n$: deve crescer devagar, como o logaritmo.

Método

1. Inteiros de 32 bits aleatórios
2. $n = 2^{16}, \ldots, 2^{22}$ (até 4 milhões)
3. Tempo: melhor de 3, dividido por n (ns por elemento)
4. Mesmos dados para os dois algoritmos

EXPERIMENTO

Tempo por elemento



n	medido	razão
65.536	149,2	9,325
131.072	157,9	9,286
262.144	167,7	9,317
524.288	179,1	9,426
1.048.576	187,5	9,373
4.194.304	206,8	9,398

razão = medido ÷ $\lg n$

✔ desvio máx. 0,4%

O radix fica em torno de 8 ns por elemento; o `qsort` passa de 100 ns, e cresce com $\lg n$.

EXPERIMENTO

O que o experimento mostrou

O tempo por elemento do radix sort é praticamente **constante** (linear no total) e mais de 10 vezes menor que o do `qsort`, cujo custo por elemento cresce como $\lg n$. O limite $\Omega(n \lg n)$ não se aplica porque o radix sort **não compara chaves**: usa os bits delas.

Parte da diferença é constante: `qsort` chama uma função de comparação por ponteiro a cada passo. Compare também com o seu quicksort da complemento 1.

Reproduza: `gcc -O2 compl4_experimento.c -o exp -lm && ./exp`



Conexões

- GPUs
- Bancos de dados
- Bioinformática
- Processamento de imagens

CONEXÕES

Onde isso aparece na computação

GPUs

Radix sort é o padrão para ordenar em GPU (CUB, Thrust): as contagens e somas de prefixo paralelizam muito bem.

Bancos de dados

Ordenar por chaves inteiras (datas, códigos) com radix e agrupar por contagem (`GROUP BY`) aparecem em bancos colunares como DuckDB.

Bioinformática

Arranjos de sufixos de genomas são construídos com radix sort sobre o alfabeto $\{A, C, G, T\}$: base 4, chaves curtas.

Processamento de imagens

O histograma de uma imagem é a primeira etapa da contagem; a equalização de histograma usa exatamente as somas acumuladas de C.

PARA CASA

Para casa

Exercícios

1. (8.2-1) Ilustre a contagem em $A = \langle 6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2 \rangle$.
2. (8.2-3) Se o último laço andar de 1 até n, o algoritmo ainda funciona? Ainda é estável?
3. (8.3-1) Ilustre o radix em COW, DOG, SEA, RUG, ROW, MOB, BOX, TAB, BAR, EAR, TAR, DIG, BIG, TEA, NOW, FOX.
4. (8.3-4) Ordene n inteiros em $0..n^3 - 1$ em tempo $O(n)$.
5. Varie a base do experimento (16, 256, 65.536) e explique o resultado pelo lema 8.4 e pelo cache.

Leitura
Cormen et al., seções 8.2 e 8.3.

Código
josedihego.net/estruturas